



Convolution Operation

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes



The Convolution Operation

Filters/Kernels, Stride, Padding এবং Feature Map Generation

বিষয়: CNN-এর মূল অপারেশন — Convolution কীভাবে কাজ করে

স্র: Deep Learning — CNN Series (পর্ব ২/৩)

পূর্বশর্ত: Introduction to CNN & Image Processing

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

ভাবো তুমি একটি ছবিতে বিড়াল খুঁজছো। তুমি কীভাবে খোঁজো?

তুমি পুরো ছবিতে চোখ বুলাও না। বরং তুমি **নির্দিষ্ট features** খোঁজো:

- 👁️ বিড়ালের চোখের আকৃতি আছে?
- 👂 কান কি ত্রিভুজ আকারের?
- 🐾 থাবার মতো কিছু দেখা যাচ্ছে?

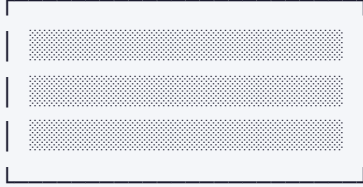
Convolution এই একই কাজ করে!

এটি একটি ছোট "চোখ" (Filter/Kernel) দিয়ে পুরো ছবিটি স্ক্যান করে এবং বিভিন্ন features খোঁজো।

🔍 বাস্তব জীবনের উদাহরণ – রান্নাঘরে ছাঁচ দিয়ে বিস্কুট কাটা!

কল্পনা করো তুমি বিস্কুট তৈরি করছো:

আটার মিষ্টি চাদর (ইনপুট ইমেজ):



বিস্কুটের ছাঁচ (Filter/Kernel):



← তারা আকৃতির ছাঁচ

ছাঁচটি পুরো চাদরে বুলিয়ে দাও → অনেক তারা বিস্কুট পাবে!

এখানে:

- আটার চাদর = Input Image (ছবি)
- বিস্কুটের ছাঁচ = Filter/Kernel
- বিস্কুট কাটার ফলাফল = Feature Map

Convolution ঠিক এভাবেই কাজ করে – একটি ছোট Filter পুরো ছবিতে স্লাইড করে এবং প্রতিটি জায়গায় "সেই feature আছে কিনা" পরীক্ষা করে!

এটি কোন সমস্যা সমাধান করে?

সমস্যা	Convolution কীভাবে সমাধান করে
ছবি অনেক বড় (লক্ষ লক্ষ pixel)	ছোট Filter দিয়ে local features খোঁজে
Feature যেকোনো জায়গায় থাকতে পারে	Filter পুরো ছবিতে স্লাইড করে
বিভিন্ন ধরনের feature দরকার	একাধিক Filter ব্যবহার করা যায়
ANN-এ অনেক বেশি parameter লাগে	Filter-এর weight share হয় (Parameter Sharing)

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ Filter / Kernel কী?

Filter (বা **Kernel**) হলো একটি ছোট number matrix। যেমন একটি 3×3 Filter:

1	0	-1
2	0	-2
1	0	-1

এটি একটি Sobel Edge Detection Filter
(উল্লম্ব edge খুঁজে বের করে)

Filter-এর বৈশিষ্ট্য:

- এটি ছোট (সাধারণত 3×3 বা 5×5)
- এর মধ্যে **Learnable Weights** আছে — Training-এর সময় এগুলো update হয়
- একটি CNN-এ **অনেক Filter** থাকে — প্রতিটি আলাদা feature খোঁজে
- ছোট Filter = **কম computation** + **বেশি efficiency**

২.২ Convolution Operation — ধাপে ধাপে

ধাপ ১: 5×5 ইনপুট ইমেজ নাও:

ইনপুট (5×5):

1	2	3	0	1
4	5	6	1	0
7	8	9	2	1
1	2	3	4	0
0	1	2	3	1

ধাপ ২: 3×3 Filter নাও:

Filter (3×3):

1	0	-1
1	0	-1
1	0	-1

ধাপ ৩: Filter-কে ইনপুটের উপর রাখো (top-left corner থেকে শুরু):

ইনপুটের প্রথম 3×3 অংশ:

1	2	3
4	5	6
7	8	9

⊗

Filter:

1	0	-1
1	0	-1
1	0	-1

ধাপ ৪: Element-wise গুণ করো এবং সব যোগ করো:

$$\begin{aligned}
 &(1 \times 1) + (2 \times 0) + (3 \times -1) + \\
 &(4 \times 1) + (5 \times 0) + (6 \times -1) + \\
 &(7 \times 1) + (8 \times 0) + (9 \times -1) \\
 \\
 &= 1 + 0 + (-3) + 4 + 0 + (-6) + 7 + 0 + (-9) \\
 &= 1 - 3 + 4 - 6 + 7 - 9 \\
 &= -6
 \end{aligned}$$

এই -6 সংখ্যাটি Feature Map-এর প্রথম cell!

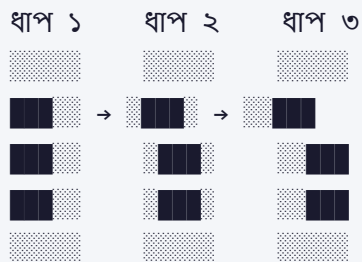
ধাপ ৫: Filter এক ধাপ ডানে সরিও (Stride=1) এবং আবার গণনা করো।

এভাবে পুরো ছবি স্ক্যান হয় → **Feature Map** তৈরি হয়!

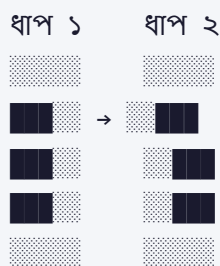
২.৩ Stride কী?

Stride মানে Filter কতটুকু এক ধাপে সরবে।

Stride = 1 (প্রতি ধাপে 1 pixel সরে):



Stride = 2 (প্রতি ধাপে 2 pixel সরে):



Stride বাড়ালে:

- ✓ Output ছোট হয় → কম computation

- ☒ Downsampling হয় (মোটী দাগে feature দেখা যায়)
- ☒ তথ্য কম ধরা পড়ে (কিছু pixel skip হয়)

২.৪ Padding কী এবং কেন দরকার?

সমস্যা: Padding ছাড়া Filter প্রতিবার সরলে Output ছোট হয়ে যায়।

Without Padding: 5×5 ইনপুট + 3×3 Filter $\rightarrow 3 \times 3$ Output (ছোট!)

With Padding: 5×5 ইনপুটের চারদিকে ০ বর্ডার যোগ করো $\rightarrow 7 \times 7$ বানাও
তারপর 3×3 Filter দিলে $\rightarrow 5 \times 5$ Output (একই সাইজ!)

Padding এর দুটি প্রধান ধরন:

১. Valid Padding (No Padding):

ইনপুট: 5×5

Output: 3×3 (ছোট হয়ে যায়)

কোনো border যোগ হয় না।

২. Same Padding (Zero Padding):

ইনপুটের চারদিকে ০ দিয়ে border:

	0	0	0	0	0	0	0	
	0	[1	2	3	0	1]	0	
	0	[4	5	6	1	0]	0	
	0	[7	8	9	2	1]	0	
	0	[1	2	3	4	0]	0	
	0	[0	1	2	3	1]	0	
	0	0	0	0	0	0	0	

7×7 padded ইনপুট

Output: 5×5 (ইনপুটের মতোই!)

Padding-এর সুবিধা:

- ছবির কিনারার তথ্য হারায় না
- Deep Network-এ সাইজ ধরে রাখা যায়
- Edge features ভালোভাবে detect হয়

২.৫ Feature Map কী?

Feature Map (বা **Activation Map**) হলো Convolution-এর output।

একটি Filter → একটি Feature Map

অনেক Filter → অনেক Feature Map (Stack করলে 3D Volume)

Filter 1 (Edge Detector) → Feature Map 1 (edges দেখায়)

Filter 2 (Corner Detector) → Feature Map 2 (corners দেখায়)

Filter 3 (Texture Detector) → Feature Map 3 (texture দেখায়)

...

Filter N → Feature Map N

এই N টি Feature Map stack করলে পাই:

Output Volume (Height × Width × N)

৩. Math / Theory**৩.১ Convolution-এর Mathematical Definition**

2D Discrete Convolution:

$$(I \circledast K)[i, j] = \sum_m \sum_n I[i+m, j+n] \times K[m, n]$$

প্রতিটি symbol মানে:

- I = Input image matrix

- K = Kernel/Filter matrix
- i, j = Output-এর position (row, column)
- m, n = Filter-এর position
- $\otimes$ = Cross-correlation (CNN-এ সাধারণত এটিই ব্যবহার হয়)
- Σ = Summation (সব গুণফলের যোগ)

৩.২ Output Size বের করার সূত্র

$$\text{Output} = \left\lfloor \frac{I + 2P - F}{S} \right\rfloor + 1$$

($\lfloor \rfloor$ = Floor function, মানে ভাগের পূর্ণ সংখ্যা)

যেখানে:

- I = Input size (width বা height)
- F = Filter size (width বা height)
- P = Padding (border-এ যোগ করা শূন্যগুলোর সংখ্যা)
- S = Stride

৩.৩ Manual Calculation — উদাহরণ সহ

উদাহরণ ১: Valid Padding

$$\begin{aligned} I &= 5, F = 3, P = 0, S = 1 \\ \text{Output} &= \left\lfloor \frac{(5 + 2 \times 0 - 3)}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{(5 - 3)}{1} \right\rfloor + 1 \\ &= \lfloor 2 \rfloor + 1 \\ &= 2 + 1 \\ &= 3 \end{aligned}$$

✓ Output: 3×3

উদাহরণ ২: Same Padding

$I = 5, F = 3, P = 1, S = 1$

$$\begin{aligned} \text{Output} &= \lfloor (5 + 2 \times 1 - 3) / 1 \rfloor + 1 \\ &= \lfloor (5 + 2 - 3) / 1 \rfloor + 1 \\ &= \lfloor 4 \rfloor + 1 \\ &= 4 + 1 \\ &= 5 \end{aligned}$$

✓ Output: 5×5 (input-এর মতোই!)

উদাহরণ ৩: Stride = 2

$I = 6, F = 3, P = 0, S = 2$

$$\begin{aligned} \text{Output} &= \lfloor (6 + 2 \times 0 - 3) / 2 \rfloor + 1 \\ &= \lfloor 3 / 2 \rfloor + 1 \\ &= \lfloor 1.5 \rfloor + 1 \\ &= 1 + 1 \\ &= 2 \end{aligned}$$

✓ Output: 2×2 (অনেক ছোট!)

৩.৪ Same Padding-এর জন্য কতটুকু Padding দরকার?

$$P = (F - 1) / 2 \quad [\text{যখন } S = 1 \text{ এবং } F \text{ বিজোড় সংখ্যা}]$$

উদাহরণ:

- 3×3 Filter → $P = (3-1)/2 = 1$
- 5×5 Filter → $P = (5-1)/2 = 2$
- 7×7 Filter → $P = (7-1)/2 = 3$

৩.৫ Parameter Count

একটি Convolutional Layer-এ মোট parameters:

$\text{Parameters} = (F \times F \times C_{\text{in}}) \times C_{\text{out}} + C_{\text{out}} (\text{bias})$

যেখানে:

- F = Filter size (e.g., 3)
- C_{in} = Input channels (e.g., RGB = 3)
- C_{out} = Output channels = Filter-এর সংখ্যা
- শেষের C_{out} = Bias terms

উদাহরণ:

32 টি 3×3 Filter, 3-channel input:

$= (3 \times 3 \times 3) \times 32 + 32$

$= 27 \times 32 + 32$

$= 864 + 32$

$= 896$ parameters মাত্র!

(ANN-এ একই কাজে লক্ষ লক্ষ parameter লাগতো!)

8. Code Example (Python)

```

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

# -----
# PART 1: Convolution হাতে করে বোঝা (NumPy)
# -----

def manual_convolution(image, kernel, stride=1, padding=0):
    """
    হাতে Convolution করার ফাংশন
    image: 2D numpy array (input image)
    kernel: 2D numpy array (filter)
    stride: কতটুকু এক ধাপে সরবে
    padding: কতটুকু border যোগ করবো
    """
    # Padding যোগ করো (চারদিকে ০ বর্ডার)
    if padding > 0:
        image = np.pad(image, padding, mode='constant', constant_values=0)

    # ইনপুট ও কার্নেলের সাইজ
    I_h, I_w = image.shape          # ইনপুটের height ও width
    K_h, K_w = kernel.shape         # কার্নেলের height ও width

    # আউটপুট সাইজ বের করো
    O_h = (I_h - K_h) // stride + 1 # আউটপুটের height
    O_w = (I_w - K_w) // stride + 1 # আউটপুটের width

    # আউটপুট ম্যাট্রিক্স তৈরি করো (শূন্য দিয়ে)
    output = np.zeros((O_h, O_w))

    # Convolution শুরু করো
    for i in range(O_h):            # প্রতিটি row-এর জন্য
        for j in range(O_w):        # প্রতিটি column-এর জন্য
            # ইনপুটের সংশ্লিষ্ট অংশ বের করো
            patch = image[
                i * stride : i * stride + K_h, # row range
                j * stride : j * stride + K_w  # column range
            ]
            # Element-wise গুণ করে যোগ করো (Dot product)
            output[i, j] = np.sum(patch * kernel)

```

```

    return output

# -----
# PART 2: বিভিন্ন Filter দিয়ে পরীক্ষা করো
# -----

# একটি সহজ 5x5 "ছবি" তৈরি করো (কৃত্রিম)
sample_image = np.array([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
], dtype=float)

print("="*50)
print("ইনপুট ইমেজ (5x5):")
print(sample_image)
print(f"সাইজ: {sample_image.shape}")

# Filter 1: Vertical Edge Detection (উল্লম্ব কিনারা খোঁজা)
vertical_edge_filter = np.array([
    [ 1,  0, -1],
    [ 1,  0, -1],
    [ 1,  0, -1]
], dtype=float)

# Filter 2: Horizontal Edge Detection (আনুভূমিক কিনারা খোঁজা)
horizontal_edge_filter = np.array([
    [ 1,  1,  1],
    [ 0,  0,  0],
    [-1, -1, -1]
], dtype=float)

# Filter 3: Blur/Smoothing Filter (ঝাপসা করা)
blur_filter = np.ones((3, 3), dtype=float) / 9 # গড় বের করে

print("\n" + "="*50)
print("Filter 1 – Vertical Edge Detection:")
print(vertical_edge_filter)

# -----

```

```
# PART 3: Valid Padding দিয়ে Convolution
# -----

result_valid = manual_convolution(
    image=sample_image,
    kernel=vertical_edge_filter,
    stride=1,
    padding=0 # কোনো padding নেই (Valid)
)

print("\n" + "="*50)
print("Valid Padding (padding=0, stride=1) এর ফলাফল:")
print(result_valid)
print(f"আউটপুট সাইজ: {result_valid.shape}")
print("→ 5×5 ইনপুট + 3×3 Filter → 3×3 Output (সাইজ কমেছে!)")

# -----
# PART 4: Same Padding দিয়ে Convolution
# -----

result_same = manual_convolution(
    image=sample_image,
    kernel=vertical_edge_filter,
    stride=1,
    padding=1 # 1 pixel border যোগ (Same padding)
)

print("\n" + "="*50)
print("Same Padding (padding=1, stride=1) এর ফলাফল:")
print(result_same)
print(f"আউটপুট সাইজ: {result_same.shape}")
print("→ 5×5 ইনপুট + 3×3 Filter → 5×5 Output (সাইজ একই!)")

# -----
# PART 5: Stride = 2 দিয়ে Convolution
# -----

result_stride2 = manual_convolution(
    image=sample_image,
    kernel=vertical_edge_filter,
    stride=2, # প্রতি ধাপে 2 pixel সরে
    padding=0
)
```

```

print("\n" + "="*50)
print("Stride=2 (padding=0, stride=2) এর ফলাফল:")
print(result_stride2)
print(f"আউটপুট সাইজ: {result_stride2.shape}")
print("→ 5×5 ইনপুট + 3×3 Filter + Stride 2 → 2×2 Output (অনেক ছোট!)")

# -----
# PART 6: Output Size সূত্র যাচাই করে
# -----

def calculate_output_size(I, F, P, S):
    """Output সাইজ বের করার ফাংশন"""
    return (I + 2 * P - F) // S + 1

print("\n" + "="*50)
print("Output Size সূত্র যাচাই:")
print(f"Valid (I=5,F=3,P=0,S=1): {calculate_output_size(5,3,0,1)} × {calculate_output_size(5,3,0,1)}")
print(f"Same (I=5,F=3,P=1,S=1): {calculate_output_size(5,3,1,1)} × {calculate_output_size(5,3,1,1)}")
print(f"Stride(I=5,F=3,P=0,S=2): {calculate_output_size(5,3,0,2)} × {calculate_output_size(5,3,0,2)}")

# -----
# PART 7: TensorFlow/Keras দিয়ে Real Example
# -----

print("\n" + "="*50)
print("TensorFlow Conv2D Layer উদাহরণ:")

# একটি Convolutional Layer তৈরি করে
conv_layer = keras.layers.Conv2D(
    filters=32,          # 32 টি ভিন্ন Filter
    kernel_size=(3, 3), # প্রতিটি Filter 3×3
    strides=(1, 1),     # Stride 1
    padding='same',     # Same padding
    activation='relu'   # ReLU Activation
)

# Dummy input তৈরি করে (1 image, 28×28, 1 channel)
dummy_input = tf.random.normal([1, 28, 28, 1])

# Layer দিয়ে পাস করে
output = conv_layer(dummy_input)

```

```
print(f"Input shape: {dummy_input.shape} → (batch, height, width, channels)")
print(f"Output shape: {output.shape} → (batch, height, width, filters)")
print(f"\nParameter count: {conv_layer.count_params()}")
print(f"গণনা:  $(3 \times 3 \times 1) \times 32 + 32 = \{(3 \times 3 \times 1 \times 32) + 32\}$  parameters")
```

Expected Output:

=====

ইনপুট ইমেজ (5×5):

```
[[0. 0. 0. 0. 0.]
 [0. 1. 1. 1. 0.]
 [0. 1. 1. 1. 0.]
 [0. 1. 1. 1. 0.]
 [0. 0. 0. 0. 0.]]
```

সাইজ: (5, 5)

=====

Valid Padding (padding=0, stride=1) এর ফলাফল:

```
[[ -1.  0.  1.]
 [ -1.  0.  1.]
 [ -1.  0.  1.]]
```

আউটপুট সাইজ: (3, 3)

→ 5×5 ইনপুট + 3×3 Filter → 3×3 Output (সাইজ কমেছে!)

=====

Same Padding (padding=1, stride=1) এর ফলাফল:

```
[[ 0. -1.  0.  1.  0.]
 [ 0. -1.  0.  1.  0.]
 [ 0. -1.  0.  1.  0.]
 [ 0. -1.  0.  1.  0.]
 [ 0. -1.  0.  1.  0.]]
```

আউটপুট সাইজ: (5, 5)

→ 5×5 ইনপুট + 3×3 Filter → 5×5 Output (সাইজ একই!)

=====

Stride=2 এর ফলাফল:

```
[[ -1.  1.]
 [ -1.  1.]]
```

আউটপুট সাইজ: (2, 2)

=====

TensorFlow Conv2D Layer উদাহরণ:

Input shape: (1, 28, 28, 1) → (batch, height, width, channels)

Output shape: (1, 28, 28, 32) → (batch, height, width, filters)

Parameter count: 320

গণনা: $(3 \times 3 \times 1) \times 32 + 32 = 320$ parameters

🔗 🎨 Visual / Diagram

৫.১ Convolution Operation-এর পূর্ণ Flow

INPUT IMAGE
(5×5)

	1		2		3		0		1	
	4		[5]		[6]		[1]		0	
	7		[8]		[9]		[2]		1	
	1		2		3		4		0	
	0		1		2		3		1	

⊗

FILTER
(3×3)

	1		0		-1	
	1		0		-1	
	1		0		-1	

→

OUTPUT (Feature Map)
(3×3)

	?			

[] = Filter যেখানে বসে আছে

৫.২ Stride Visualization

STRIDE = 1: (Dense Scanning)



(3 ধাপ → 3×3 output)

STRIDE = 2: (Skip Scanning)



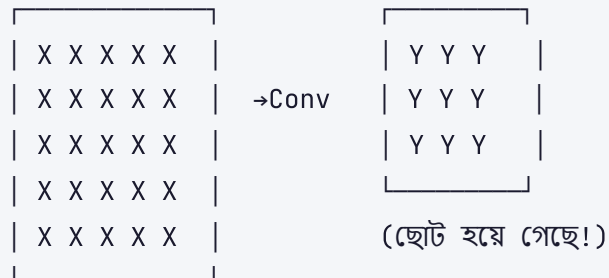
(2 ধাপ → 2×2 output, দ্রুত কিন্তু কম detail)

৫.৩ Padding Visualization

 VALID PADDING (No Padding):

Input: 5×5

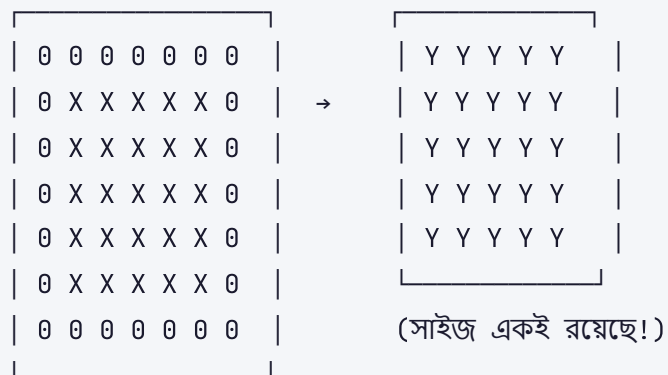
Output: 3×3



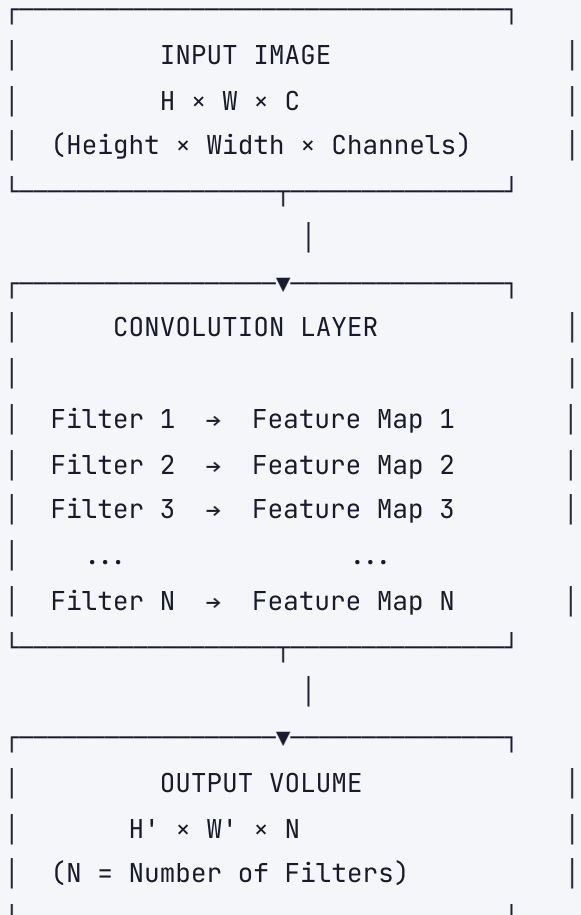
 SAME PADDING (Zero Padding, P=1):

Padded Input: 7×7

Output: 5×5



4.8 Multiple Filters → Feature Maps (3D Volume)



৫.৫ CNN-এ Feature Hierarchy (কীভাবে Deep Layers কাজ করে)

Layer 1: Simple Features



Layer 2: Combinations



Layer 3: Complex Parts



Layer 4: Full Objects



৬. ✅ Real-world Use Cases

Use Case ১ — Instagram Face Filter 🎨

কোম্পানি: Meta (Instagram/Facebook)

কীভাবে: Convolution দিয়ে মুখের features (চোখ, নাক, মুখ) detect করে, তারপর তার উপরে AR filter বসায়।

Filter ব্যবহার: Edge detection → facial landmark detection

Use Case ২ — Google Photos স্বয়ংক্রিয় Album 📷

কোম্পানি: Google

কীভাবে: CNN দিয়ে ছবি থেকে বিভিন্ন feature map তৈরি করে মানুষ, স্থান, জিনিস চিনতে পারে এবং স্বয়ংক্রিয়ভাবে album সাজায়।

Use Case ৩ — Tesla Autopilot 🚗

কোম্পানি: Tesla

কীভাবে: গাড়ির ক্যামেরা থেকে আসা real-time ছবিতে Convolution দিয়ে রাস্তার চিহ্ন, পথচারী, অন্য গাড়ি detect করে।

স্পেশাল: Real-time inference লাগে বলে optimized stride ব্যবহার হয়।

Use Case ৪ — Medical Image Analysis 🏥

কোম্পানি: Google DeepMind, Siemens

কীভাবে: X-ray বা MRI scan-এ Convolution দিয়ে টিউমার, fracture বা অন্যান্য রোগ detect করে।

সাফল্য: কিছু ক্ষেত্রে ডাক্তারের চেয়েও নির্ভুল!

Use Case ৫ — Snapchat Dog Filter 🐶

কোম্পানি: Snap Inc.

কীভাবে: Real-time face segmentation করে Convolution দিয়ে, তারপর মুখের সঠিক অংশে কুকুরের কান ও নাক বসায়।

৭. ⚖️ Pros & Cons

সুবিধা ✅	অসুবিধা ❌
Parameter Sharing — কম parameter লাগে	বড় ছবিতে ধীর — Resolution বাড়লে computation বাড়ে
Translation Invariant — feature যেকোনো জায়গায় থাকলেও detect করে	Large Receptive Field কঠিন — দূরের parts-এর সম্পর্ক বোঝা না
Sparse Connectivity — প্রতিটি neuron শুধু local area দেখে	Rotation/Scale Sensitivity — ঘুরানো ছবি নাও চিনতে পারে
Hierarchical Feature Learning — simple থেকে complex features শেখে	Hyperparameter Tuning কঠিন — Filter size, stride, padding বেছে নেওয়া কঠিন
Efficient GPU Computation — GPU-তে দ্রুত চলে	Black Box — কোন Filter কী শিখলো বোঝা কঠিন
Proven Track Record — ImageNet-এ দারুণ সাফল্য	অনেক Data দরকার — অল্প data-তে overfit হয়

৮. ⚠️ Common Mistakes & Gotchas

ভুল ৬ – Padding ভুলে যাওয়া

```
# ❌ ভুল: Deep network-এ padding না দিলে feature map দ্রুত শেষ হয়
conv = Conv2D(filters=64, kernel_size=3) # Default: padding='valid'

# ✅ সঠিক: Same padding ব্যবহার করো Deep Network-এ
conv = Conv2D(filters=64, kernel_size=3, padding='same')
```

ভুল ২ – Filter Size সম্পর্কে ভুল ধারণা

❌ ভুল ধারণা: বড় Filter = ভালো performance
 ✅ সত্য: ছোট Filter (3×3) stack করা বড় Filter (7×7)-এর চেয়ে বেশি efficient এবং বেশি non-linearity শেখে

দুটি 3×3 filter = একটি 5×5 filter-এর receptive field
 কিন্তু parameter: $2 \times (3 \times 3) = 18$ vs $5 \times 5 = 25 \rightarrow 28\%$ কম!

ভুল ৩ – Output Size ভুল গণনা

```
# ❌ ভুল: Output size assume করা
# সবসময় সূত্র ব্যবহার করো:
output_size = (input_size + 2*padding - kernel_size) // stride + 1

# বা Keras-এ:
model.summary() # প্রতিটি layer-এর output shape দেখায়
```

ভুল ৪ – বেশি Stride দেওয়া

❌ বেশি Stride (S=4 বা 5): অনেক তথ্য হারায়
 ✅ সাধারণত Stride 1 বা 2 ব্যবহার করো
 Downsampling-এর জন্য Pooling Layer বেশি পছন্দের

ভুল ৫ – Channel ভুলে যাওয়া

- ❌ ভুল: একটি Filter একটি 2D matrix মনে করা
- ✅ সত্য: RGB ছবির জন্য Filter আসলে 3D ($3 \times 3 \times 3$)
প্রতিটি channel-এ আলাদা weight থাকে!

Input: $H \times W \times 3$ (RGB)

Filter: $3 \times 3 \times 3$ (3D kernel)

Output: একটি 2D scalar value per position

ভুল ৬ — Bias যোগ করতে ভুলে যাওয়া

Convolution-এর পর সাধারণত একটি Bias term যোগ করা হয়:

$\text{output} = (\text{input} \otimes \text{kernel}) + \text{bias}$

Keras Conv2D-এ default-এ use_bias=True থাকে ✅

৯. 🔗 Related Topics

আগে কী জানা দরকার?

- ✅ **Matrix Multiplication** — Dot product বুঝতে হবে
- ✅ **Introduction to CNN** — Pixels, RGB Channels, Image Tensors
- ✅ **ANN এবং Neural Network Basics** — Weights, Bias, Activation
- ✅ **NumPy** — Python-এ array manipulation
- ✅ **Backpropagation** — Filter weights কীভাবে আপডেট হয়

পরে কী শেখা উচিত?

- 🔴 **Pooling Layers** — Max/Average Pooling (CNN Series ৩/৩)
- 🔴 **Batch Normalization** — Training stabilize করার কৌশল
- 🔴 **Famous CNN Architectures** — LeNet, AlexNet, VGG, ResNet
- 🔴 **Transfer Learning** — Pre-trained model-এর Feature দিয়ে নিজের কাজ করা
- 🔴 **Object Detection** — YOLO, SSD (Convolution ব্যবহার করে)

-  **Dilated/Atrous Convolution** — বড় receptive field কম parameter-এ

১০. Memory Tricks

মনে রাখার সহজ কৌশল

Trick 1 — "বিস্কুটের ছাঁচ" ট্রিক:

Filter = বিস্কুটের ছাঁচ
 Image = আটার চাদর
 Feature Map = কেটে বের করা বিস্কুটগুলো
 Stride = প্রতিবার কতটুকু সরে কাটে
 Padding = চাদরের বর্ডারে অতিরিক্ত আটা

Trick 2 — Output Size সূত্র মনে রাখা:

"I + 2P - F / S + 1"

মনে রাখো: "আই-পি-এফ-এস-প্লাস-ওয়ান"

I nput

2 times Padding

F ilter

S tride

+1 always!

Trick 3 — Padding এর উদ্দেশ্য:

"P = Preserve" (সাইজ ধরে রাখো)

"V = Vanish" (সাইজ কমে যায়)

Same padding = Size Preserved ✓

Valid padding = Size Vanishes (কমে) ✓

Trick 4 — Stride effect:

Stride বাড়লে Output ছোট হয়
মনে রাখো: "বড় লাফ = ছোট Map"

🌟 ৬ লাইনে সারসংক্ষেপ:

Convolution = একটি ছোট Filter (Kernel) পুরো ছবিতে স্লাইড করে Element-wise গুণ করে ও যোগ করে Feature Map বের করে; Stride নিয়ন্ত্রণ করে Filter কতটুকু সরবে এবং Padding নিশ্চিত করে Feature Map সাইজ ঠিক থাকে।

 Quick Reference Table

Parameter	কী করে	সাধারণ মান
Filter Size	Feature detect করার area	3×3, 5×5
# of Filters	কতটি Feature Map বের হবে	32, 64, 128, 256
Stride (S)	Filter কতটুকু এক ধাপে সরে	1 (সাধারণত), 2
Valid Padding	Output ছোট হয়	Medical images, feature extraction
Same Padding	Output unchanged	Deep networks, UNet

 তৈরির তারিখ: ২০২৬-০৪-০৪

 সিরিজ: CNN — পর্ব ২/৩

 পরবর্তী নোটি: [Pooling Layers & Fully Connected Network](#)

